Efficient Implementation of Rank and Select Functions for Succinct Representation

Dong Kyue Kim[1,⋆], Joong Chae Na[2], Ji Eun Kim[1], and Kunsoo Park[2,⋆⋆]

[1] School of Electrical and Computer Engineering, Pusan National University
[2] School of Computer Science and Engineering, Seoul National University
kpark@theory.snu.ac.kr

Abstract. Succinct representation is a space-efficient method to represent n discrete objects by $O(n)$ bits. In order to access directly the ith object of succinctly represented data structures in constant time, two fundamental functions, `rank` and `select`, are commonly used. However, little efforts were made on analyzing practical behaviors of these functions despite their importance for succinct representations.

In this paper we analyze the behavior of Clark's algorithm which is the only one to support `select` in constant time using $o(n)$-bit space of extra space, and show that the performance of Clark's algorithm gets worse as the number of 1's in a bit-string becomes fewer and there exists a worst case in which a large amount of operations are needed. Then, we propose two algorithms that overcome the drawbacks of Clark's. These algorithms take constant time for `select`, and one uses $o(n)$ bits for extra space and the other uses $n + o(n)$ bits in the worst case. Experimental results show that our algorithms compute `select` faster than Clark's.

1 Introduction

To analyze performance of data structures, the processing time and the amount of used storage are measured in general. With the rapid proliferation of information, it is increasingly important to focus on the storage requirements of data structures. Traditionally, discrete objects such as elements of sets or arrays, nodes of trees, vertices and edges of graphs, and so on, are represented as integers which are the indices of elements in a consecutive memory block or values of logical addresses in main memory. If we store n discrete objects in this way, they occupy $O(n)$ words, i.e., $O(n \log n)$ bits.

Recently, a method to represent n objects by $O(n)$ bits, which is called *succinct representation*, was developed. Various succinct representation techniques have been developed to represent data structures such as sets, static and dynamic

⋆ Supported by "Research Center for Logistics Information Technology (LIT)" hosted by the Ministry of Education & Human Resources Development in Korea.
⋆⋆ Supported by MOST grant M6-0405-00-0022.

S.E. Nikoletseas (Ed.): WEA 2005, LNCS 3503, pp. 315–327, 2005.

dictionaries [7, 14] trees [11], and graphs [16, 8]. Moreover, succinct representation is also applied to permutations [10] and functions [13]. Especially, in the areas of string precessing and bio-informatics, to store efficiently enormous DNA sequence data, full-text index data structures are developed such as compressed suffix trees and arrays [12, 5, 15, 6, 4] and the FM-index [2, 3]. If we use these succinctly represented data structures, we can perform very fast pattern searching using small space.

Most succinct representations use `rank` and `select` as their basic functions. The rank and select functions are defined as follows. Given a static bit-string A,

- $\texttt{rank}_A(x)$: Counts the number of 1's up to and including the position x in A.
- $\texttt{select}_A(y)$: Finds the position of the yth 1 bit in A.

For example, if $A = 1\ 0\ 0\ 1\ 0\ 1\ 1\ 0$, $\texttt{rank}_A(5) = 2$ and $\texttt{select}_A(2) = 4$.

There have been only three results on `rank` and `select`. Jacobson [7, 8] first considered `rank` and `select` and proposed a data structure supporting the functions in $O(\log n)$ time. Jacobson constructs a two-level directory structure and performs a direct access and a binary search on the directory for `rank` and `select`, respectively. Clark [1] improved Jacobson's result to support `rank` and `select` in constant time by adding lookup-tables to complement the two-level directory of Jacobson's. Munro and Raman [11] proposed a three-level directory structure for `rank`. Recently, Miltersen [9] showed lower bounds for `rank` and `select`. For `select`, Clark's data structure is the only one to support `select` in constant time using $o(n)$-bit space of extra space. Despite the importance of `select` for succinct representation, little efforts were made on analyzing its behavior on different kinds of bit strings.

In this paper we analyze the behavior of Clark's algorithm for `select` on various bit-strings and then we get the following results.

- Its performance gets worse as the number of 1's in the bit-string becomes fewer. Especially when the number of 1's in the bit-string is very few (the portion of 1's is less than 10%), Clark's algorithm becomes quite slow.
- In Clark's algorithm, there exists a worst case in which a large number of accesses to lookup tables is needed.
- In addition, it is difficult to implement Clark's algorithm using a byte-based method because the block sizes of Clark's algorithm are varying.

We propose two algorithms that resolve these drawbacks of Clark's algorithm by transforming the given bit-string A into a bit-string where 1's are distributed quite regularly. Our algorithms support `rank` and `select` in constant time, and Algorithm I uses $o(n)$ bits for extra space and Algorithm II uses $n + o(n)$ bits in worst case. However, Algorithm II works fast and uses less space than Algorithm I in practice. Both algorithms have the following properties.

- Our algorithms are not affected by the distribution of 0's and 1's because we partition a bit-string into blocks of constant size to make a directory for `select`. Hence, the algorithms take uniform retrieval time.

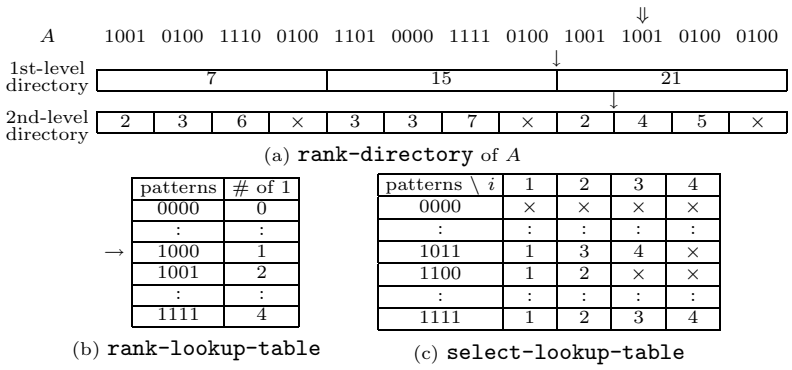


Fig. 1. Examples of data structures

- Only a small number of table accesses is always required in our algorithms.
- Our algorithms are easy to implement because directories are composed of constant-sized blocks. Thus, our algorithms can be implemented by a byte-based method which is suitable for modern general-purpose computers.

In experimental results, we show the behavior of Clark's algorithm and show that Algorithm I and Algorithm II resolve the drawbacks of Clark's algorithm. We compare the performance of Clark's algorithm, Algorithm I and II. Our algorithms construct and compute `select` faster than Clark's. Algorithm II using byte-based implementation shows remarkable performance in computing `select`.

2 Preliminaries

We first give definitions of some data structures that will be used for `rank` and `select` later. Let A be a static bit-string of length n. We denote the ith bit by $A[i]$ and the substring $A[i]A[i+1]\cdots A[j]$ by $A[i..j]$. For simplicity, we assume that $\sqrt{\log n}$, $\log n$ and $\log^2 n$ are integers.

We first define a hierarchical directory structure for `rank`. Given a bit-string A, we define `rank-directory` of A as the following two-level directory (see Fig. 1 (a)):

- We partition A into *big blocks* of size $\log^2 n$. Each big block of the 1st-level directory records the accumulated number of 1's from the first big block. That is, the ith entry contains the number of 1's in $A[1..i\log^2 n]$ for $1 \le i \le \lfloor n/\log^2 n \rfloor$.
- We partition A into *small blocks* of size $\log n$. Each small block of the 2nd-level directory records the accumulated number of 1's from the first small block within each big block. That is, the ith entry contains the number of 1's in $A[i'\log^2 n + 1..i\log n]$ for $1 \le i \le \lfloor n/\log n \rfloor$, where $i' = \lfloor i\log n/\log^2 n \rfloor$.

If we have `rank-directory` of A, we can get $\mathtt{rank}_A(i)$ for an ending position i of every small block in constant time. For example, $\log^2 n = 16$ and $\log n = 4$

in Fig. 1 (a). Then we can find $\texttt{rank}_A(36) = 17$ by adding the value 15 in the 2nd entry of the 1st-level directory and the value 2 in the 9th entry of the 2nd-level directory, which give the number of 1's in $A[1..32]$ and $A[33..36]$, respectively.

Lemma 1. *Given a bit-string A of length n,* `rank-directory` *of A can be stored in $o(n)$ bits.*

We define some lookup tables that enable us to compute `rank` and `select` in constant time. For some integer $c > 1$ ($c = 2$ suffices), `rank-lookup-table` is the table where an entry contains the number of 1's in each possible bit pattern of length $(\log n)/c$. Similarly, `select-lookup-table` is the table where an entry contains the position of the ith 1 bit in each possible bit pattern of length $(\log n)/c$, for $1 \leq i \leq \log n/c$. Figure 1 (b) and (c) show `rank-lookup-table` and `select-lookup-table` for patterns of length 5, respectively.

Lemma 2. *Both* `rank-lookup-table` *and* `select-lookup-table` *can be stored in $o(n)$ bits.*

3 Behavior Analysis of Clark's Algorithm

In this section we describe Clark's algorithms [1] for `rank` and `select`, and analyze the behavior of Clark's algorithm for `select`.

3.1 Clark's Algorithm

We first describe the algorithm for `rank` and then describe the algorithm for `select`. For $\texttt{rank}_A$, Clark used `rank-directory` of A and `rank-lookup-table`. To get $\texttt{rank}_A(x)$, one first computes $\texttt{rank}_A(i)$ for an ending position i of the small block which includes $A[x]$ using `rank-directory`, and then counts the number of 1's in remaining x mod $\log n$ bits, which can be found by adding at most c entries in `rank-lookup-table` after masking out unwanted trailing bits.

Example 1. For bit-string A in Fig. 1, we want to compute $\texttt{rank}_A(38)$. We first get $\texttt{rank}_A(36) = 17$ using `rank-directory` in Fig. 1 (a). We get a bit-pattern 1000 by mask out $A[37..40] = 1001$ with 1100 and get the number of 1's in 1000 using `rank-lookup-table` in Fig. 1 (b). So, we get $\texttt{rank}_A(38) = 18$ by adding 1 to 17.

For $\texttt{select}_A$, Clark used `rank-lookup-table`, `select-lookup-table`, and the following multi-level directory which is stored in $o(n)$ bits.

- The 1st-level directory records the position of every $(\log n \log\log n)$'th 1 bit. That is, the ith entry contains $\texttt{select}_A(i \log n \log\log n)$ for $1 \leq i \leq \lfloor n/\log n \log\log n \rfloor$.
- The 2nd-level directory consists of two kinds of tables. Let r be the size of a subrange between two adjacent values in the 1st-level directory and consider this range of size r.

  - If $r \geq (\log n \log\log n)^2$, then all answers of `select` in this range are recorded explicitly using $\log n$ bits for each entry.
  - Otherwise, one re-subdivides the range and records the position, relative to the start of the range, of every $(\log r \log\log n)$'th 1 bit.
- Let r' be the size of a subrange between two adjacent values in the 2nd-level directory.
  - If $r' \geq \log r' \log r (\log\log n)^2$, then all answers are recorded explicitly.
  - Otherwise, one records nothing. In this case, `rank-lookup-table` and `select- lookup-table` are used.

Now we describe how to get $\texttt{select}_A(y)$ using the data structures above. To get $\texttt{select}_A(y)$, one first finds the correct 1st-level directory entry, at position $\lfloor y/\log n \log\log n \rfloor$, and compute r. If $r \geq (\log n \log\log n)^2$, one can find the value of $\texttt{select}_A(y)$ in the 2nd-level directory. Otherwise, a similar search is performed in the 2nd-level directory resulting in either finding the answer from the 3rd-level directory or scanning a small number of bits using lookup-tables. It was proven that the length of a bit-string scanned using lookup-tables is less than $16(\log\log n)^4$ [1]. So one can perform `select` on a range of $16(\log\log n)^4$ bits using a constant number of accesses to the lookup-tables.

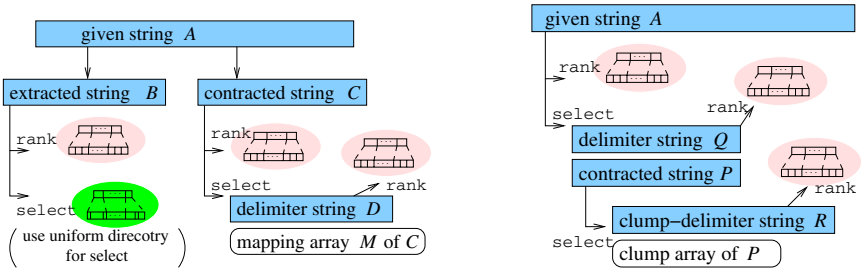
3.2 Behavior of Clark's Algorithm

We analyze the behavior of Clark's algorithm for `select` on various bit-strings and then we get the followings.

- The retrieval time of Clark's algorithm for `select` varies according to distribution of 0's and 1's in bit-strings. The performance gets worse as the number of 1's in the bit-string becomes fewer. Especially when the number of 1's in the bit-string is very few (the portion of 1's is less than 10%), Clark's algorithm becomes quite slow.
- In Clark's algorithm, there exists a worst case in which a large number of accesses to lookup-tables is needed. The reason is that $16(\log\log n)^4$ is much larger than $\log n$ in practice although the former is asymptotically smaller than the latter. For example, in case that the length of a bit-string is 2^{32}, the size of a block for accessing tables is 4096 and the number of table accesses is 256 in the worst case. The number 256 is too big to regard as a constant in practice.
- The structure of directories is irregular because the subranges r and r' of entries vary. It makes it difficult to implement the algorithm using a byte-based implementation method.

4 Algorithm I

In this section we present our first algorithm that overcomes the drawbacks of Clark's algorithm for `select`. This algorithm also adopts the approach using



(a) Algorithm I (b) Algorithm II

Fig. 2. Structures of Algorithms I and II

multi-level directories and lookup-tables. The difficulty of developing an algorithm for `select` results from the irregular distribution of 1's. While Clark's algorithm overcomes the difficulty by classifying the subranges of directories into two groups: dense one and sparse one, we do it by transforming A into a bit-string where 1's are distributed quite regularly.

4.1 Definitions

Given a bit-string S of length m, we divide S into blocks of size b. There are two kinds of blocks. One is a block where all elements are 0 and the other is a block where there is at least one 1. We call the former *zero-block (ZB)* and the latter *nonzero-block (NZ)*.

- The *contracted* string of S is defined as a bit-string S_c of length m/b such that $S_c[i] = 0$ if the ith block of S is a zero-block, $S_c[i] = 1$ otherwise.
- The *extracted* string of S is defined as a bit-string S_e which is formed by concatenating nonzero-blocks of S in order. Hence, the length of S_e is m in the worst case, and the distance between the ith 1 bit and the jth 1 bit is at most $(j - i + 1)b - 1$ for $i < j$.
- The *delimiter* string of S is defined as a bit-string S_d such that $S_d[i] = 0$ if the ith 1 bit and the $(i-1)$st 1 bit of S are contained in the same block, and $S_d[i] = 1$ otherwise. We define $S_d[1] = 1$. Note that the length of S_d is equal to the number of 1's in S and so it is m in the worst case. The value of $\texttt{rank}_{S_d}(i)$ means the number of nonzero blocks up to the block (including itself) containing the ith 1 bit of S.

Example 2. Bit-strings S, S_c, S_e and S_d. We assume that we divide S into blocks of size 4.

S_c	1				0				1				1			
S	0	1	1	1	0	0	0	0	0	0	1	0	1	1	0	1
S_e	0	1	1	1					0	0	1	0	1	1	0	1
S_d		1	0	0							1		1	0		0

4.2 Skeleton of Algorithm I

Let B be the extracted string of A, and C be the contracted string of A when dividing A into blocks of size $\sqrt{\log n}$. We transform A into B and C, and compute $\texttt{rank}_A$ and $\texttt{select}_A$ using B and C. We first consider the properties of B and C and describe how to compute $\texttt{rank}_A$ and $\texttt{select}_A$ using $\texttt{rank}$'s and $\texttt{select}$'s of B and C. In the next section we present algorithms and data structures for $\texttt{rank}$'s and $\texttt{select}$'s of B and C.

Recall that blocks of B contain at least one 1 bit, and thus the distance between the ith 1 bit and the jth 1 bit in B is at most $(j-i+1)\sqrt{\log n}-1$ for $i<j$. Bit-string C represents a mapping between A and B. The length of C is $n/\sqrt{\log n}$ and the length of B is n in the worst case. See Fig. 2 (a).

Example 3. Bit-strings A, B and C. We assume that $\sqrt{\log n}=5$.

						↓	⇓
A	01101	00000	00000	11010	00000	01001	00100
C	1	0	0	1	0	1	1
B	01101	11010	01001	00100			
			↑	⇑			

We describe how to compute $\texttt{rank}_A(x)$ and $\texttt{select}_A(y)$ using B and C. For computing $\texttt{rank}_A(x)$, we find the index x' of B which corresponds to $A[x]$ and compute $\texttt{rank}_B(x')$. Let x_b be the block number of A which contains $A[x]$ and x_p be the position of $A[x]$ in the x_bth block of A, that is, $x_b=\lceil x/\sqrt{\log n}\,\rceil$ and $x_p=x-(x_b-1)\sqrt{\log n}$. Then,

$$x'=\texttt{rank}_C(x_b-1)\times\sqrt{\log n}+x_p\times C[x_b] \quad \text{and} \quad \texttt{rank}_A(x)=\texttt{rank}_B(x').$$

For computing $\texttt{select}_A(y)$, we compute $\texttt{select}_B(y)$ and find an index of A which corresponds to $B[\texttt{select}_B(y)]$. Let s_b be the block number of B which contains the yth 1 bit, that is, $s_b=\lceil \texttt{select}_B(y)/\sqrt{\log n}\,\rceil$. Then,

$$\texttt{select}_A(y)=\texttt{select}_B(y)+(\texttt{select}_C(s_b)-s_b)\sqrt{\log n}.$$

Example 4. For A of Example 3, suppose that we want to compute $\texttt{rank}_A(28)$ and $\texttt{select}_A(9)$. For $\texttt{rank}_A(28)$, we get $x_b=6$ and $x_p=3$, that is, $A[28]$ is the 3rd bit in the 6th block of A. Because $\texttt{rank}_C(5)=2$, we get $x'=2\times5+3=13$, so $\texttt{rank}_A(28)=\texttt{rank}_B(13)=6$. For $\texttt{select}_A(9)$, we get $\texttt{select}_B(9)=18$, so $s_b=4$. Because $\texttt{select}_C(4)=7$, we get $\texttt{select}_A(9)=18+(7-4)\times5=33$.

4.3 Ranks and Selects for B and C

For $\texttt{rank}_B$, we build $\texttt{rank-directory}$ of B and $\texttt{rank-lookup-table}$. For $\texttt{rank}_C$, we build $\texttt{rank-directory}$ of C and $\texttt{rank-lookup-table}$. We can compute $\texttt{rank}_B(x)$ and $\texttt{rank}_C(x)$ using these data structures in constant time as in Section 2.

Select for extracted string B. For $\texttt{select}_B$, we use lookup-tables and a two-level directory which is a little different from $\texttt{rank-directory}$.

- The 1st-level directory records the position of every $\log^2 n$'th 1 bit in B. This directory has at most $n/\log^2 n$ entries and each entry requires $\log n$ bits. Thus the space of this directory is $n/\log^2 n \times \log n = o(n)$ bits.
- The 2nd-level directory records the position of every $\sqrt{\log n}$'th 1 bit in the ranges of 1st-level directory. This directory has at most $n/\sqrt{\log n}$ entries and each entry requires $\log(\log^2 n \times \sqrt{\log n})$ bits because all blocks of size $\sqrt{\log n}$ have at least one 1 bit. Thus the space of this directory is $n/\sqrt{\log n} \times 5/2 \times \log\log n = o(n)$ bits.
- We also maintain `rank-lookup-table` and `select-lookup-table`.

We can compute $\texttt{select}_B(y)$ by accessing the directory and the lookup-tables as in `rank`. Because the 2nd-level directory records the position of every $\sqrt{\log n}$'th 1 bit, we need to scan a substring of at most length $\sqrt{\log n} \times \sqrt{\log n}$ using the lookup-tables. It can be done by accessing the lookup-tables at most c times.

Select for contracted string C. We use a different approach for $\texttt{select}_C$ because the range of contiguous 0's is not bounded in C. Let D be the delimiter string of C when dividing C into blocks of size $\log n$. The length of D is $n/\sqrt{\log n}$ in the worst case. See Example 5. We construct the following auxiliary data structures.

- We construct a data structure for $\texttt{rank}_D$. It consists of `rank-directory` of D and `rank-lookup-table`. The value of $\texttt{rank}_D(i)$ means the number of nonzero blocks of size $\log n$ up to the block (including itself) containing the ith 1 bit of C.
- We construct an array M whose ith entry represents the block number of the ith nonzero-block in C. We call M the *mapping* array of C. This array has at most $n/(\log n\sqrt{\log n})$ entries and each entry requires $\log(n/(\log n\sqrt{\log n}))$ bits. Thus the space of this array is $n/(\log n\sqrt{\log n}) \times \log\left(n/(\log n\sqrt{\log n})\right) = o(n)$bits.

Example 5. Bit-strings C and D, and the mapping array M of C. We assume that $\log n = 4$.

C	0 1 1 1	0 0 0 0	0 0 1 0	1 1 0 1
D	1 0 0		1	1 0 0

i	1	2	3
M	1	3	4

In order to get $\texttt{select}_C(y)$, we first compute the block number containing the yth 1 bit in C using $\texttt{rank}_D$ and array M. Then, we can find the position of the yth 1 bit in C by scanning the bit-string of this block using the lookup-tables.

Example 6. Suppose that we want to find $\texttt{select}_C(6)$. The `rank` of the 6th bit in D is 3 and the value of the 3rd entry in array M is 4. Thus, the 6th 1 bit is in the 4th block of C. We can also know that the first 3 blocks of C contains four 1 bits using $\texttt{rank}_C$. The final job is to find the 2nd 1 bit in the 4th block using the lookup-tables.

Theorem 1. *Algorithm I performs* $\texttt{rank}_A$ *and* $\texttt{select}_A$ *in constant time using* $o(n)$ *bits of extra space.*

5 Algorithm II

In this section we describe Algorithm II, which is simpler and more practical than Algorithm I but theoretically uses $n + o(n)$ bits of extra space. We also propose a new implementation method which is suitable for modern general-purpose computers.

5.1 Description of Algorithm II

We only describe the algorithm for $\texttt{select}_A$ since the algorithm for $\texttt{rank}_A$ is the same as Clark's. For $\texttt{select}_A$, we use an approach similar to that used for $\texttt{select}_C$ in Section 4.3. When dividing A into blocks of size $\log n$, let P and Q be the contracted string and the delimiter string of A, respectively. The length of P is $n/\log n$ and the length of Q is n in the worst case.

We can compute $\texttt{select}_A(y)$ using $\texttt{rank}_Q$ and $\texttt{select}_P$. Because $\texttt{rank}_Q(y)$ gives the number of nonzero blocks up to the block (including itself) containing the yth 1 bits of A and $\texttt{select}_P(k)$ represents the block number of the kth nonzero block, $\texttt{select}_P(\texttt{rank}_Q(y))$ is the block number containing yth 1 bit in A. Then, we can find the position of the yth 1 bits in A by scanning the bit-string of this block using the lookup-tables. For $\texttt{rank}_Q$, we build `rank-directory` of Q. For $\texttt{select}_P$, we use a new approach.

We describe an approach for $\texttt{select}_P$ which uses small space in practice. We call a bundle of contiguous 0's a *clump*. In Example 7, there are 4 clumps in P. We define the *clump-delimiter* string R of P as follows: $R[i] = 0$ if the ith 1 bit of P is adjacent to the $(i-1)$st 1 bit, $R[i] = 1$ otherwise. We define $R[1]$ as 0 if $P[1] = 1$, and 1 otherwise. See Example 7. The length of R is $n/\log n$ in the worst case. We construct the following auxiliary data structures.

- We construct a data structure for $\texttt{rank}_R$. The value of $\texttt{rank}_R(i)$ means how many clumps there are in front of the ith 1 bit of P. In Example 7, there are 3 clumps in front of the 7th 1 bit.
- We construct an array where the ith entry represents the accumulated number of 0's up to the ith clump (including itself) in P. We call it the *clump* array of P. In practice, most elements of P are 1 because P is a delimiter string. Therefore, there are a few clumps in P and so the size of this table is very small. Note that we do not need to maintain bit-string P.

Example 7. Bit-strings P and R, and the clump array of P.

P:	0	0	1	1	1	0	1	1	0	0	1	1	1	1	0	1	
R:			1	0	0		1	0			1	0	0	0		1	

The clump array of P

index	1	2	3	4
	2	3	5	6

In order to get $\texttt{select}_P(i)$, we first know how many clumps there are in front of the ith 1 bit of P using $\texttt{rank}_R(i)$ and compute the number j of 0's in front of the ith 1 bit using the clump array . Then $\texttt{select}_P(i) = i + j$.

Theorem 2. *Algorithm II performs* $\texttt{rank}_A$ *and* $\texttt{select}_A$ *in constant time using* $o(n)$ *and* $n + o(n)$ *bits of extra space, respectively.*

5.2 Byte-Based Implementation of Algorithm II

We present an efficient implementation method of Algorithm II. The directories and lookup-tables are based on bits while atomic units in modern computers are not bits but bytes. Therefore, we need *bit-operations* such as *bitwise-and, bitwise-or*, and *shift* in order to get the values of entries. It causes inefficiency in time. One method avoiding such inefficiency is to allocate space to entries by the units of bytes. For example, we allocate 2 bytes to an entry which requires 12 bits. However, this method may waste much space. We present a byte-based implementation method reducing waste of space and avoiding bit-operations. This method is applied to Q and R as well as A. We assume that n is less than 2^{32}, the maximum value represented by a word in 32-bit machines. However, our method can be extended to apply to the case of $n \geq 2^{32}$.

The key idea of our method is that ranges of directories and lookup-tables are adjusted to multiples of 8. The following data structure, which we use to implement Algorithm II, is an instance of our implementation method.

- The 1st-level directory contains `rank` for every multiple of 2^8. Each entry requires at most 32 bits. Particularly, the ranges of 1st-level directory is adjusted to 2^k, where k is a multiples of 8 in order to allocate space to entries of 2nd-level directory by byte units.
- The 2nd-level directory contains `rank`, for every multiple of 2^5, within the subranges of size 2^8. Each entry requires 8 bits.
- For each possible bit pattern of length 8, the rank-lookup-table gives the number of 1's in the pattern. Each entry requires 3 bits but we allocate 8 bits for each entry in order to avoid bit-operations. We may access the rank-lookup-table four times to get `rank`.

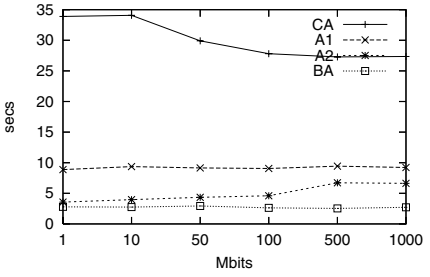
Because each entry in each directory is stored in bits of a multiple of 8, we can find values of entries without bit-operations. So retrievals in our method are fast.

6 Experimental Results

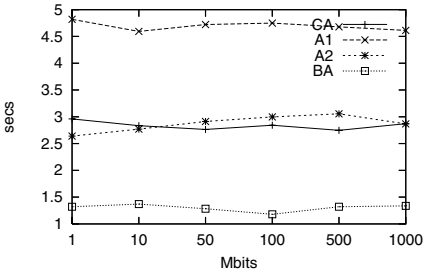
We present experimental results comparing Clark's algorithm (CA) with Algorithm I (A1), Algorithm II (A2), and byte-based Algorithm II (BA). All algorithms except algorithm BA are allowed to use bit-operations.

We used Microsoft Visual C++ 6.0 to implement the algorithms and performed these experiments on the 2.8Ghz Pentium IV with 4GB main memory. We measured retrieval time of `rank` and `select`, construction time, and space of auxiliary data structures. We performed experiments on bit-strings where the ratio of 1's is about 3 % and random bit-strings.

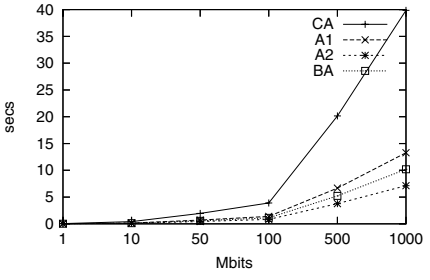
Experiment 1 (bit-strings with 3 % 1's)**:** This experiment shows the drawbacks of Clark's algorithm pointed out in Section 3.2. Figure 3 (a) and (b) show retrieval time of `select` and `rank`, respectively. In these figures, the vertical axis represents the time taken to perform 10^7 random queries and the horizontal axis represents the length of bit-string A. In `select`, our algorithms are about $3 \sim 10$



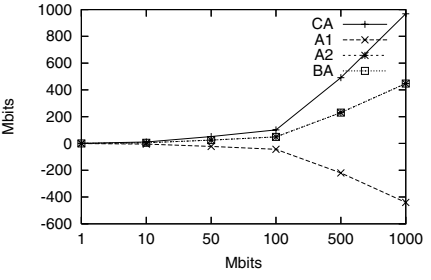
(a) `select` retrieval time



(b) `rank` retrieval time

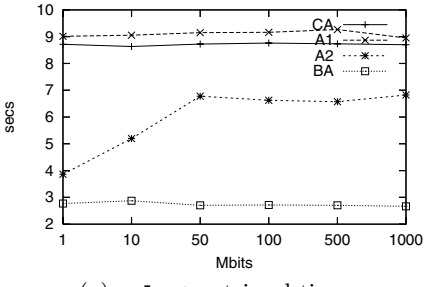


(c) construction time

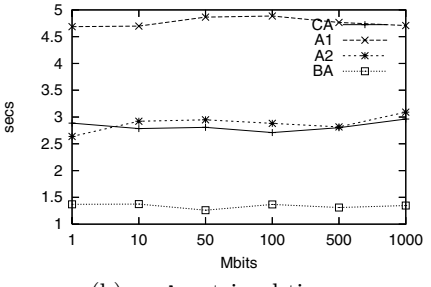


(d) space

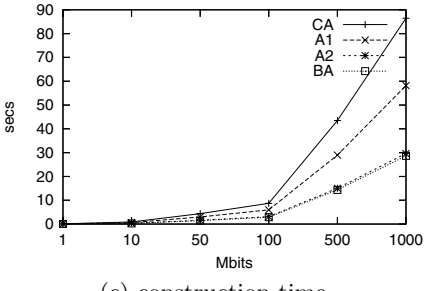
Fig. 3. Experimental results on bit-strings with 3 % 1's



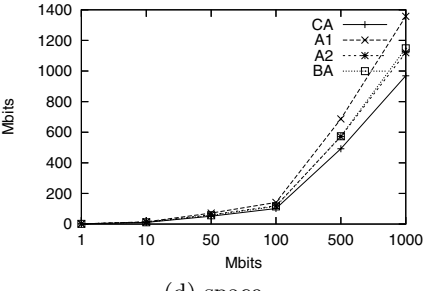
(a) `select` retrieval time



(b) `rank` retrieval time



(c) construction time



(d) space

Fig. 4. Experimental results on random bit-strings

times faster than Clark's. In `rank`, Algorithm I is the slowest. The reason is that it needs two `rank`'s. Algorithm II and Clark's algorithm have the same performance in `rank` because two algorithms for `rank` are the same. Figure 3 (c) shows construction time. Clark's algorithm is the slowest due to the complication and irregularity of the data structures. Our algorithms are about $3 \sim 5$ times faster than Clark's. Figure 3 (c) shows space of auxiliary data structures. We do not count the space for a given string A. Clark's algorithm requires the most space. The reason why Algorithm I has negative values is that a transformed string B is even shorter than given string A.

Experiment 2 (random bit-strings): This experiment shows general performance of the algorithms. Figure 4 (a) and (b) show retrieval time of `select` and `rank`, respectively. The fact that byte-based Algorithm II is the fastest in both tells efficiency of byte-based implementation in time. In `select`, the performance of Algorithm I and Clark's algorithm are similar and they are the slowest. The reason why Clark's Algorithm is slow is that it needs many accesses to lookup-tables. In every algorithm, retrieval time of `select` is slower than that of `rank`. Figure 4 (c) and (d) show construction time and space of auxiliary data structures, respectively. Our algorithms are about $1.5 \sim 2.5$ times faster than Clark's. Clark's algorithm requires less space than ours. However, the difference on space is not large compared to the differences on retrieval time and construction time.

References

1. D. R. Clark. *Compact Pat Trees*. PhD thesis, University of Waterloo, Waterloo, 1988.
2. P. Ferragina and G. Manzini. Opportunistic data structures with applications. *FOCS'00*, pages 390–398, 2000.
3. P. Ferragina and G. Manzini. An experimental study of an opportunistic index. *SODA'01*, pages 269–278, 2001.
4. R. Grossi, A. Gupta, and J. S. Vitter. High-order entropy-compressed text indexes. *SODA'03*, pages 841–850, 2003.
5. R. Grossi and J. S. Vitter. Compressed suffix arrays and suffix trees with applications to text indexing and string matching. *STOC'00*, pages 397–406, 2000.
6. W. K. Hon, K. Sadakane, and W. K. Sung. Breaking a time-and-space barrier in constructing full-text indices. *FOCS'03*, pages 251–260, 2003.
7. G. Jacobson. *Succinct Static Data Structures*. PhD thesis, Carnegie Mellon University, Pittsburgh, 1988.
8. G. Jacobson. Space-efficient static trees and graphs. *FOCS'89*, pages 549–554, 1989.
9. P. B. Miltersen. Lower bounds on the size of selection and rank indexes. *SODA'05*, pages 11–12, 2005.
10. J. I. Munro, R. Raman, V. Raman, and S. S. Rao. Succinct representations of permutations. *ICALP'03*, pages 345–356, 2003.
11. J. I. Munro and V. Raman. Succinct representation of balanced parentheses and static trees. *SIAM J. on Comp.*, 31(3):762–776, 2001.
12. J. I. Munro, V. Raman, and S.S. Rao. Space efficient suffix trees. *J. Alg.*, 39(2):205–222, 2001.

13. J. I. Munro and S. S. Rao. Succinct representations of functions. *ICALP'04*, pages 1006–1015, 2004.
14. R. Raman and S. S. Rao. Succinct dynamic dictionaries and trees. *ICALP'03*, pages 357–368, 2003.
15. K. Sadakane. Succinct representations of lcp information and improvements in the compressed suffix arrays. *SODA'02*, pages 225–232, 2002.
16. G. Turan. Succinct representations of graphs. *Disc. App. Math.*, 8:289–294, 1984.